

Euclidean algorithms (Basic and Extended)

Last Updated : 17 Feb, 2025



The Euclidean algorithm is a way to find the greatest common divisor of two positive integers. GCD of two numbers is the largest number that divides both of them. A simple way to find GCD is to **factorize** both numbers and multiply common prime factors.

Examples:

input: $a = 12, b = 20$

Output: 4

*Explanation: The Common factors of (12, 20) are 1, 2, and 4 and **greatest is 4.***

input: $a = 18, b = 33$

Output: 3

Explanation: The Common factors of (18, 33) are 1 and 3 and greatest is 3.

$$36 = 2 \times 2 \times 3 \times 3$$

$$60 = 2 \times 2 \times 3 \times 5$$

??

$$\begin{aligned} \text{GCD} &= \text{Multiplication of common factors} \\ &= 2 \times 2 \times 3 \\ &= 12 \end{aligned}$$

Table of Content

- [Basic Euclidean Algorithm for GCD](#)
- [Extended Euclidean Algorithm](#)
- [How does Extended Algorithm Work?](#)
- [How is Extended Algorithm Useful?](#)

Basic Euclidean Algorithm for GCD

The algorithm is based on the below facts.

- If we subtract a smaller number from a larger one (we reduce a larger number), GCD doesn't change. So if we keep subtracting repeatedly the larger of two, we end up with GCD.
- Now instead of subtraction, if we divide the larger number, the algorithm stops when we find the remainder 0.

CPP

C

Java

Python

C#

JavaScript

```
// C++ program to demonstrate working of
// extended Euclidean Algorithm
#include <bits/stdc++.h>
using namespace std;

// Function to return
// gcd of a and b
int findGCD(int a, int b) {
    if (a == 0)
        return b;
    return findGCD(b % a, a);
}

int main() {
    int a = 35, b = 15;
    int g = findGCD(a, b);
    cout << g << endl;
    return 0;
}
```

Output

```
GCD(10, 15) = 5
GCD(35, 10) = 5
GCD(31, 2) = 1
```

Time Complexity: $O(\log \min(a, b))$

Auxiliary Space: $O(\log (\min(a, b)))$

Extended Euclidean Algorithm

Extended Euclidean algorithm also finds integer coefficients x and y such that: $ax + by = \gcd(a, b)$

Examples:

Input: $a = 30, b = 20$

Output: $\gcd = 10, x = 1, y = -1$

Explanation: $30*1 + 20*(-1) = 10$

Input: $a = 35, b = 15$

Output: $\gcd = 5, x = 1, y = -2$

Explanation: $35*1 + 15*(-2) = 5$

The extended Euclidean algorithm updates the results of $\gcd(a, b)$ using the results calculated by the recursive call $\gcd(b\%a, a)$. Let values of x and y calculated by the recursive call be x_1 and y_1 . x and y are updated using the below expressions.

$$\begin{aligned} ax + by &= \gcd(a, b) \\ \gcd(a, b) &= \gcd(b\%a, a) \\ \gcd(b\%a, a) &= (b\%a)x_1 + ay_1 \\ ax + by &= (b\%a)x_1 + ay_1 \\ ax + by &= (b - [b/a] * a)x_1 + ay_1 \\ ax + by &= a(y_1 - [b/a] * x_1) + bx_1 \end{aligned}$$

Comparing LHS and RHS,

$$x = y_1 - [b/a] * x_1$$
$$y = x_1$$

C++

C

Java

Python

C#

JavaScript

```
// C++ program to demonstrate working of
// extended Euclidean Algorithm
#include <bits/stdc++.h>
using namespace std;

// Function for extended Euclidean Algorithm
int gcdExtended(int a, int b, int &x, int &y) {

    // Base Case
    if (a == 0) {
```

```

        x = 0;
        y = 1;
        return b;
    }

    int x1, y1;
    int gcd = gcdExtended(b%a, a, x1, y1);

    // Update x and y using results of
    // recursive call
    x = y1 - (b/a) * x1;
    y = x1;
    return gcd;
}

int findGCD(int a, int b) {

    int x = 1, y = 1;
    return gcdExtended(a, b, x, y);
}

int main() {
    int a = 35, b = 15;
    int g = findGCD(a, b);
    cout << g << endl;
    return 0;
}

```

Output

5

Time Complexity: $O(\log \min(a, b))$

Auxiliary Space: $O(\log (\min(a,b)))$

How does Extended Algorithm Work?

As seen above, x and y are results for inputs a and b,

$$a.x + b.y = gcd \quad \text{-----(1)}$$

And x_1 and y_1 are results for inputs $b\%a$ and a
 $(b\%a).x_1 + a.y_1 = \text{gcd}$

When we put $b\%a = (b - (\lfloor b/a \rfloor).a)$ in above,
we get following. Note that $\lfloor b/a \rfloor$ is floor(b/a)

$$(b - (\lfloor b/a \rfloor).a).x_1 + a.y_1 = \text{gcd}$$

Above equation can also be written as below

$$b.x_1 + a.(y_1 - (\lfloor b/a \rfloor).x_1) = \text{gcd} \quad \text{---(2)}$$

After comparing coefficients of 'a' and 'b' in (1) and (2), we get following,

$$x = y_1 - \lfloor b/a \rfloor * x_1$$

$$y = x_1$$

How is Extended Algorithm Useful?

The extended Euclidean algorithm is particularly useful when a and b are coprime (or gcd is 1). Since x is the modular multiplicative inverse of "a modulo b", and y is the modular multiplicative inverse of "b modulo a". In particular, the computation of the modular multiplicative inverse is an essential step in RSA public-key encryption method.

 Comment

K kartik

 300 

Article Tags :

Mathematical

DSA

GCD-LCM

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information